

DOS Chain — v6 Validator Registration

Off-Chain BLS Signing & P-Chain Submission

Date	February 13, 2026
Chain	DOS Chain (Blockchain ID: 22v7AG7h6qaVxd4bLvAsSsg2LZ4RCn5iVYgFn7a2Fj1LCuYwjv)
Subnet	nQCwF6V9y8VFjvMuPeQVWYn6ba75518Dpf6ZMWZNb3NyTA94
Network	Avalanche Mainnet
Target Validator	v6 — NodeID—Mu35k31HAXFbEp1SqJr1uUrue9nwqVFe6
Prepared by	Ava Labs Solutions Engineering

Contents

1. Background & Problem Statement
 2. Pre-Check: Is v6 Already Registered?
 3. Scenario A — Reactivate v6 (Balance Top-Up)
 4. Scenario B — Register v6 (Off-Chain BLS Signing)
 5. Go Script: Complete RegisterL1ValidatorTx Tool
 6. Post-Registration: Warp Timestamp Fix
 7. Verification & Monitoring
- Appendix A: Required Information Checklist
- Appendix B: Security Considerations

1. Background & Problem Statement

The DOS Chain is stuck at block **2,273,699**. All 4 validators (v0, v1, v2, v6) bootstrap successfully and enter consensus, but no new blocks are produced. Debug logs confirm:

```
DEBUG <22v7AG7... Chain> snowman/engine.go:631 processing block not found  
{"requestedHeight": 2273700, "lastAcceptedHeight": 2273699}
```

The root cause involves v6 not being active in the P-Chain validator set. Without v6, the remaining validators may not meet the **67% weight threshold** required for consensus, which prevents block production.

Why Normal Registration Doesn't Work

Normal Flow (requires block production):

```
ValidatorManager.initializeValidatorRegistration() → Warp message emitted on-chain → Aggregator collects BLS signatures via P2P → RegisterL1ValidatorTx on P-Chain
```

Since the chain can't produce blocks, Step 1 can't execute. The `initializeValidatorRegistration()` transaction gets accepted into the mempool but never mined.

The Bypass: Off-Chain BLS Signing

Off-Chain Flow (no block production needed):

```
Construct RegisterL1ValidatorMessage manually → Sign with BLS private keys from v0, v1, v2 → Aggregate signatures off-chain → RegisterL1ValidatorTx on P-Chain
```

The P-Chain only checks that the Warp message has valid BLS signatures from $\geq 67\%$ of the L1's validator weight. It does **not** verify the message originated from an on-chain transaction. We can construct and sign the message entirely off-chain using the validators' BLS private keys (`signer.key` files).

2. Pre-Check: Is v6 Already Registered?

Do this first!

If v6 is already registered but ran out of balance, the fix is a simple P-Chain transaction (Scenario A) and you can skip the complex off-chain signing entirely.

Run this on any running validator:

```
curl -s -X POST --data '{
  "jsonrpc": "2.0", "id": 1,
  "method": "platform.getCurrentValidators",
  "params": {"subnetID": "nQCwF6V9y8VFjvMuPeQVWYn6ba75518Dpf6ZMWZNb3NyTA94"}
}' -H 'content-type: application/json' http://127.0.0.1:9650/ext/bc/P | \
jq '.result.validators[] | {nodeID, weight, balance}'
```

Look for `NodeID-Mu35k31HAXFbEp1SqJr1uUrue9nwqVFe6` in the output.

Result	Meaning	Action
v6 listed, balance: 0	v6 ran out of AVAX and was disabled	Scenario A — Simple balance top-up
v6 NOT listed at all	v6 was never registered post-conversion	Scenario B — Off-chain signing & registration
v6 listed, balance > 0	v6 is active; issue is elsewhere	Investigate further — contact Ava Labs

Also check ALL validators' balances.

If any of v0, v1, or v2 also have balance: 0, they are disabled too. ALL validators need positive balances to participate in consensus.

3. Scenario A — Reactivate v6 (Balance Top-Up)

SIMPLE FIX If v6 is already registered but has `balance: 0`:

`IncreaseL1ValidatorBalanceTx` is a P-Chain-only transaction. No L1 block production is required.

Using avalanche-cli

```
# Install avalanche-cli (if not already installed)
curl -sSfL https://raw.githubusercontent.com/ava-labs/avalanche-cli/main/scripts/install.sh | sh -s

# Fund v6 with 10 AVAX
avalanche validator increaseBalance \
  --node-id NodeID-Mu35k31HAXFbEp1SqJr1uUrue9nwqVFe6 \
  --balance 10 \
  --mainnet

# Also check and fund v0, v1, v2 if their balances are low
avalanche validator increaseBalance \
  --node-id NodeID-3K3PUAqo3cKxRoQyYto1EsXtuTHoDZ2B6 \
  --balance 10 --mainnet

avalanche validator increaseBalance \
  --node-id NodeID-Ma3Ztm2A48bCVjfiSmoZS5MKNzKVTRN7j \
  --balance 10 --mainnet

avalanche validator increaseBalance \
  --node-id NodeID-G8yv8mWQy8FLdFfAhJ4v294T5Z2DWDVEH \
  --balance 10 --mainnet
```

Cost:

~1.33 AVAX/month per validator. 10 AVAX provides ~7.5 months of runway per validator.

After funding, proceed to **Section 6: Warp Timestamp Fix**.

4. Scenario B — Register v6 (Off-Chain BLS Signing)

ADVANCED Only needed if v6 is NOT in the P-Chain validator set at all.

4.1 Prerequisites

Requirement	Details
Go 1.22+	Install from https://go.dev/dl/
v6 BLS public key & proof of possession	From <code>info.getNodeID</code> on v6
<code>signer.key</code> files from v0, v1, v2	Located at <code>~/.avalanchego/staking/signer.key</code> on each validator
P-Chain funded key	At least 11 AVAX (10 deposit + 1 fees)
ValidatorManager proxy address	The contract address on the DOS chain
Network access	To a mainnet Avalanche node on port 9650

4.2 Gather Information

A. Get v6's BLS credentials:

```
# Run on v6's machine
curl -s -X POST --data '{
  "jsonrpc":"2.0","id":1,
  "method":"info.getNodeID"
}' -H 'content-type:application/json' http://127.0.0.1:9650/ext/info | jq
'.result'

# Output includes:
#   "nodeID": "NodeID-Mu35k31HAXFbEp1SqJr1uUrue9nwqVFe6"
#   "nodePOP": {
#     "publicKey": "0x...",          <-- v6 BLS public key (48 bytes)
#     "proofOfPossession": "0x..." <-- v6 BLS PoP (96 bytes)
#   }
```

B. Get canonical validator set ordering:

```
curl -s -X POST --data '{
  "jsonrpc":"2.0","id":1,
  "method":"platform.getCurrentValidators",
  "params":{"subnetID":"nQCwF6V9y8VFjvMuPeQVWYn6ba75518Dpf6ZMWZNb3NyTA94"}
}' -H 'content-type:application/json' http://127.0.0.1:9650/ext/bc/P | \
jq '.result.validators[] | {nodeID, weight}'
```

C. Copy `signer.key` files securely:

```
# From each validator machine, copy to a secure working directory
scp user@v0-host:~/avalanchego/staking/signer.key ./keys/v0-signer.key
scp user@v1-host:~/avalanchego/staking/signer.key ./keys/v1-signer.key
scp user@v2-host:~/avalanchego/staking/signer.key ./keys/v2-signer.key
```

Security Warning:

`signer.key` files are the BLS private keys for each validator. They are critical secrets equivalent to validator identity. Transfer over SSH/SCP only. Delete all copies from the working machine after the registration is complete.

4.3 How It Works

The script performs these steps:

1. **Construct** a `RegisterL1ValidatorMessage` with v6's details (NodeID, BLS public key, weight, expiry, balance/disable owners)
2. **Wrap** it in an `AddressedCall` payload with the ValidatorManager address as the source
3. **Create** an `UnsignedMessage` with the DOS Chain's blockchain ID and mainnet network ID
4. **Sign** the unsigned message with BLS private keys from v0, v1, v2
5. **Aggregate** the BLS signatures into a `BitSetSignature` with correct signer indices
6. **Submit** `RegisterL1ValidatorTx` on the P-Chain with the signed warp message + v6's proof of possession + AVAX deposit

5. Go Script: RegisterL1ValidatorTx Tool

5.1 Project Setup

```
mkdir dos-register-validator && cd dos-register-validator  
mkdir keys
```

Create `go.mod` :

```
module dos-register-validator  
  
go 1.22  
  
require github.com/ava-labs/avalanchego v1.12.2
```

5.2 main.go

Create `main.go` with the following content. Fill in the values marked with `<-- FILL IN` .

```

package main

import (
    "context"
    "encoding/hex"
    "fmt"
    "log"
    "sort"
    "time"

    "github.com/ava-labs/avalanchego/ids"
    "github.com/ava-labs/avalanchego/utils/constants"
    "github.com/ava-labs/avalanchego/utils/crypto/bls"
    "github.com/ava-labs/avalanchego/utils/crypto/bls/signer/localsigner"
    "github.com/ava-labs/avalanchego/utils/set"
    "github.com/ava-labs/avalanchego/units"
    "github.com/ava-labs/avalanchego/vms/platformvm/warp"
    warpmessage "github.com/ava-labs/avalanchego/vms/platformvm/warp/
message"
    warppayload "github.com/ava-labs/avalanchego/vms/platformvm/warp/
payload"
    "github.com/ava-labs/avalanchego/wallet/subnet/primary"
)

func main() {
    ctx := context.Background()

    // =====
    // CONFIGURATION – FILL IN ALL VALUES BELOW
    // =====

    networkID := constants.MainnetID // 1

    blockchainID, _ := ids.FromString(
        "22v7AG7h6qaVxd4bLvAsSsg2LZ4RCn5iVYgFn7a2Fj1LCuYwjv",
    )
    subnetID, _ := ids.FromString(
        "nQCwF6V9y8VFjvMuPeQVWwYn6ba75518Dpf6ZMWZNb3NyTA94",
    )

    // ValidatorManager proxy contract address on the DOS chain
    managerAddressHex := "0x..." // <-- FILL IN

    // v6 details (from info.getNodeID on v6)
    v6NodeIDStr      := "NodeID-Mu35k31HAXFbEp1SqJr1uUrue9nwqVFe6"
    v6BLSPubKeyHex   := "..."    // <-- FILL IN: nodePOP.publicKey
    v6BLSPoPHex      := "..."    // <-- FILL IN:
nodePOP.proofOfPossession

    // Weight for v6 (match existing validators)
    v6Weight := uint64(20)        // <-- FILL IN

    // AVAX deposit for v6 on the P-Chain (10 AVAX = ~7.5 months)
    depositBalance := 10 * units.Avox

    // BLS private key files from the SIGNING validators (v0, v1, v2)
    // Copy from ~/.avalanchego/staking/signer.key on each machine

```

```

signerKeyPaths := []string{
    "./keys/v0-signer.key",
    "./keys/v1-signer.key",
    "./keys/v2-signer.key",
}

// Node IDs of the signing validators (SAME ORDER as key paths)
signerNodeIDs := []string{
    "NodeID-3K3PUAqo3cKxRoQyYto1EsXtuTHoDZ2B6",
    "NodeID-Ma3Ztm2A48bCVjfiSmoZS5MKNzKVTRN7j",
    "NodeID-G8yv8mWQy8FLdFfAhJ4v294T5Z2DWDVEH",
}

// P-Chain private key (hex) – needs ~11 AVAX balance
pChainPrivKeyHex := "..." // <-- FILL IN

// Avalanche node RPC endpoint
nodeURI := "http://127.0.0.1:9650"

// P-Chain owner address (receives remaining deposit on removal)
ownerAddressHex := "..." // <-- FILL IN (20-byte short ID, hex)

// =====
// END CONFIGURATION
// =====

fmt.Println("=== DOS Chain v6 Validator Registration ===")
fmt.Println("    Off-Chain BLS Signing Approach")
fmt.Println()

// --- Parse v6 details ---
v6NodeID, err := ids.NodeIDFromString(v6NodeIDStr)
if err != nil {
    log.Fatalf("Parse v6 NodeID: %v", err)
}

v6BLSPubKeyBytes, _ := hex.DecodeString(strip0x(v6BLSPubKeyHex))
var v6BLSPubKey [bls.PublicKeyLen]byte
copy(v6BLSPubKey[:], v6BLSPubKeyBytes)

v6BLSPoPBytes, _ := hex.DecodeString(strip0x(v6BLSPoPHex))
var v6PoP [bls.SignatureLen]byte
copy(v6PoP[:], v6BLSPoPBytes)

managerAddr, _ := hex.DecodeString(strip0x(managerAddressHex))

ownerAddrBytes, _ := hex.DecodeString(strip0x(ownerAddressHex))
var ownerAddr ids.ShortID
copy(ownerAddr[:], ownerAddrBytes)

balanceOwner := warpmessage.PChainOwner{
    Threshold: 1,
    Addresses: []ids.ShortID{ownerAddr},
}

disableOwner := warpmessage.PChainOwner{
    Threshold: 1,
    Addresses: []ids.ShortID{ownerAddr},
}

```

```

}

expiry := uint64(time.Now().Add(24 * time.Hour).Unix())

// =====
// STEP 1: Construct RegisterL1Validator message
// =====
fmt.Println("[1/5] Constructing RegisterL1Validator message...")

registerMsg, err := warpmessage.NewRegisterL1Validator(
    subnetID, v6NodeID, v6BLSPubKey,
    expiry, balanceOwner, disableOwner, v6Weight,
)
if err != nil {
    log.Fatalf("Create message: %v", err)
}

validationID := registerMsg.ValidationID()
fmt.Printf("    ValidationID: %s\n", validationID)
fmt.Printf("    Expiry:      %s\n",
    time.Unix(int64(expiry), 0).UTC().Format(time.RFC3339))

// Wrap in AddressedCall
addrCall, err := warppayload.NewAddressedCall(
    managerAddr, registerMsg.Bytes(),
)
if err != nil {
    log.Fatalf("Create AddressedCall: %v", err)
}

// Create unsigned warp message
unsignedWarp, err := warp.NewUnsignedMessage(
    networkID, blockchainID, addrCall.Bytes(),
)
if err != nil {
    log.Fatalf("Create unsigned warp: %v", err)
}

fmt.Printf("    Warp msg hash: %s\n", unsignedWarp.ID())

// =====
// STEP 2: Determine canonical validator ordering
// =====
fmt.Println("\n[2/5] Determining canonical validator ordering...")

var allNodeIDs []ids.NodeID
for _, nidStr := range signerNodeIDs {
    nid, _ := ids.NodeIDFromString(nidStr)
    allNodeIDs = append(allNodeIDs, nid)
}

// Sort by NodeID bytes (canonical P-Chain ordering)
sort.Slice(allNodeIDs, func(i, j int) bool {
    return allNodeIDs[i].Compare(allNodeIDs[j]) < 0
})

nodeIDToIndex := make(map[ids.NodeID]int)

```

```

    for i, nid := range allNodeIDs {
        fmt.Printf("        [%d] %s\n", i, nid)
        nodeIDToIndex[nid] = i
    }

    // =====
    // STEP 3: Load BLS keys and sign the warp message
    // =====
    fmt.Println("\n[3/5] Loading BLS keys and signing...")

    signerBits := set.NewBits()
    var blsSigs []*bls.Signature

    for i, keyPath := range signerKeyPaths {
        sk, err := localsigner.FromFile(keyPath)
        if err != nil {
            log.Fatalf("Load key %s: %v", keyPath, err)
        }

        nid, _ := ids.NodeIDFromString(signerNodeIDs[i])
        idx := nodeIDToIndex[nid]

        sig, err := sk.Sign(unsignedWarp.Bytes())
        if err != nil {
            log.Fatalf("Sign with %s: %v", keyPath, err)
        }

        signerBits.Add(idx)
        blsSigs = append(blsSigs, sig)
        fmt.Printf("        Signed: %s (index %d)\n", signerNodeIDs[i],
idx)
    }

    // =====
    // STEP 4: Aggregate signatures
    // =====
    fmt.Println("\n[4/5] Aggregating BLS signatures...")

    aggSig, err := bls.AggregateSignatures(blsSigs)
    if err != nil {
        log.Fatalf("Aggregate: %v", err)
    }

    var aggSigBytes [bls.SignatureLen]byte
    copy(aggSigBytes[:], bls.SignatureToBytes(aggSig))

    signedWarp, err := warp.NewMessage(
        unsignedWarp,
        &warp.BitSetSignature{
            Signers:  signerBits.Bytes(),
            Signature: aggSigBytes,
        },
    )
    if err != nil {
        log.Fatalf("Create signed warp: %v", err)
    }

```

```

    fmt.Printf("        Signed message: %d bytes\n",
len(signedWarp.Bytes()))

    // =====
    // STEP 5: Submit RegisterL1ValidatorTx on P-Chain
    // =====
    fmt.Println("\n[5/5] Submitting RegisterL1ValidatorTx on P-Chain...")

    pKeyBytes, _ := hex.DecodeString(strip0x(pChainPrivKeyHex))
    kc := primary.NewKeychain(pKeyBytes...)
    wallet, err := primary.MakePWallet(
        ctx, nodeURI, kc,
        primary.WalletConfig{SubnetIDs: []ids.ID{subnetID}},
    )
    if err != nil {
        log.Fatalf("Create wallet: %v", err)
    }

    tx, err := wallet.IssueRegisterL1ValidatorTx(
        depositBalance, v6PoP, signedWarp.Bytes(),
    )
    if err != nil {
        log.Fatalf("Issue tx: %v", err)
    }

    fmt.Println("\n=====")
    fmt.Println("  SUCCESS")
    fmt.Println("=====")
    fmt.Printf("  TX ID:          %s\n", tx.ID())
    fmt.Printf("  ValidationID:   %s\n", validationID)
    fmt.Printf("  Deposit:        %.2f AVAX\n",
        float64(depositBalance)/float64(units.Avox))
    fmt.Println("\n  v6 is now registered on the P-Chain.")
    fmt.Println("  Proceed to Section 6: Warp Timestamp Fix.")
}

func strip0x(s string) string {
    if len(s) >= 2 && s[:2] == "0x" {
        return s[2:]
    }
    return s
}

```

5.3 Build & Run

```

# Initialize and download dependencies
go mod tidy

# Build
go build -o register-v6 .

# Run (ensure signer.key files are in ./keys/ and config is filled in)
./register-v6

```

Expiry window:

The script sets a 24-hour expiry on the registration message. You must run the script and have the P-Chain transaction confirmed within 24 hours. If it expires, simply re-run the script.

Note on validator ordering:

The script sorts the signing validators by NodeID bytes to determine canonical indices. If there are additional validators registered on the P-Chain beyond v0, v1, v2 (even if disabled), they must be included in the ordering. Run the `platform.getCurrentValidators` command in Section 2 and verify the validator list matches exactly.

6. Post-Registration: Warp Timestamp Fix

REQUIRED After v6 is registered on the P-Chain, block production still won't resume until the warp timestamp configuration is corrected on all 4 validators.

The Problem

Parameter	Current Value	Required Value
Last block timestamp	~1735776000 (Jan 1, 2026 11:59 UTC)	—
Warp activation timestamp	1767225600 (Jan 31, 2026) WRONG	1735689600 (Jan 1, 2026 00:00 UTC) BEFORE LAST BLOCK

Fix: Apply to ALL 4 Validators

```
CHAIN_ID="22v7AG7h6qaVxd4bLvAsSsg2LZ4RCn5iVYgFn7a2Fj1LCuYwjv"
CONFIG_DIR="$HOME/.avalanchego/configs/chains/$CHAIN_ID"

mkdir -p "$CONFIG_DIR"

# Backup existing configs
cp "$CONFIG_DIR/config.json" "$CONFIG_DIR/config.json.backup" 2>/dev/null
cp "$CONFIG_DIR/upgrade.json" "$CONFIG_DIR/upgrade.json.backup" 2>/dev/null

# Fix 1: Warp timestamp (MUST be before last block timestamp)
cat > "$CONFIG_DIR/upgrade.json" << 'EOF'
{
  "precompileUpgrades": [
    {
      "warpConfig": {
        "blockTimestamp": 1735689600
      }
    }
  ]
}
EOF

# Fix 2: ProposerVM config
cat > "$CONFIG_DIR/config.json" << 'EOF'
{
  "proposervm-block-delay-min": "100ms",
  "proposervm-block-delay-max": "5s",
  "proposer-min-block-delay": 0,
  "proposer-num-historical-blocks": 512,
  "state-sync-enabled": true,
  "pruning-enabled": true
}
EOF

echo "Config applied for $CHAIN_ID"
```

Sequential Restart

CRITICAL: Never restart all validators at once.

Restart one at a time with 2-minute gaps. Wait for each to become healthy before restarting the next.

```
# Restart order: v0 → v1 → v2 → v6
# For each validator:

sudo systemctl restart avalanche

# Wait 30 seconds, then verify health
sleep 30
curl -s -X POST --data '{"jsonrpc":"2.0","id":1,"method":"health.health"}' \
  -H 'content-type:application/json' http://127.0.0.1:9650/ext/health | jq
'.result.healthy'

# Verify peer count (should be >= 2)
curl -s -X POST --data '{"jsonrpc":"2.0","id":1,"method":"info.peers"}' \
  -H 'content-type:application/json' http://127.0.0.1:9650/ext/info | jq
'.result.numPeers'

# Wait until healthy=true and peers>=2, then move to next validator
```

7. Verification & Monitoring

Verify Block Production

```
CHAIN_ID="22v7AG7h6qaVxd4bLvAsSsg2LZ4RCn5iVYgFn7a2Fj1LCuYwjv"

# Check current block height
BLOCK=$(curl -s -X POST --data
'{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber"}' \
  -H 'content-type:application/json' \
  http://127.0.0.1:9650/ext/bc/$CHAIN_ID/rpc | jq -r '.result')
echo "Current block: $((16#${BLOCK:2}))"

# Wait 30 seconds and check again – block number should increase
sleep 30
BLOCK2=$(curl -s -X POST --data
'{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber"}' \
  -H 'content-type:application/json' \
  http://127.0.0.1:9650/ext/bc/$CHAIN_ID/rpc | jq -r '.result')
echo "Block after 30s: $((16#${BLOCK2:2}))"
```

Verify Pending Transaction

```
# Check if the test transaction from Feb 9 was mined
curl -s -X POST --data '{
  "jsonrpc":"2.0","id":1,"method":"eth_getTransactionByHash",
  "params":
  ["0xf64ff6a29087ef8a17694238d6e3e58cfd58383863d45746139cd274306138d2"]
}' -H 'content-type:application/json' \
  http://127.0.0.1:9650/ext/bc/$CHAIN_ID/rpc | jq '.result.blockNumber'

# If it returns a hex block number (not null), blocks are producing
```

If Blocks Still Don't Produce After 5 Minutes

Database resync may be needed. Do **one validator at a time**:

```
CHAIN_ID="22v7AG7h6qaVxd4bLvAsSsg2LZ4RCn5iVYgFn7a2Fj1LCuYwjv"

# 1. Stop the validator
sudo systemctl stop avalanchego

# 2. BACKUP staking keys (CRITICAL – do not skip)
cp -r ~/.avalanchego/staking /tmp/staking-backup-$(date +%s)

# 3. Delete only the DOS chain database
find ~/.avalanchego/db -name "$CHAIN_ID*" -type d -exec rm -rf {} + 2>/dev/null

# 4. Restart (will resync from other validators)
sudo systemctl start avalanchego

# 5. Monitor sync progress (15–60 minutes)
watch -n 10 'curl -s -X POST --data "{\"jsonrpc\":\"2.0\", \"id\":1, \
  \"method\":\"info.isBootstrapped\", \"params\":{\"chain\":\"'$CHAIN_ID'\"}}\" \
  -H \"content-type:application/json\" http://127.0.0.1:9650/ext/info | jq'
```

Appendix A: Required Information Checklist

Before proceeding with Scenario B, collect and confirm all of the following:

#	Item	Source	Value
1	v6 BLS Public Key	<code>info.getNodeID</code> on v6	<i>to be filled</i>
2	v6 BLS Proof of Possession	<code>info.getNodeID</code> on v6	<i>to be filled</i>
3	ValidatorManager proxy address	DOS chain deployment records	<i>to be filled</i>
4	v0 <code>signer.key</code>	<code>~/ .avalanchego/staking/signer.key</code> on v0	<i>secure transfer</i>
5	v1 <code>signer.key</code>	<code>~/ .avalanchego/staking/signer.key</code> on v1	<i>secure transfer</i>
6	v2 <code>signer.key</code>	<code>~/ .avalanchego/staking/signer.key</code> on v2	<i>secure transfer</i>
7	P-Chain private key (hex)	Team's key with ≥ 11 AVAX on P-Chain	<i>secure handling</i>
8	P-Chain owner address	For remaining balance & disable rights	<i>to be filled</i>
9	v6 desired weight	Should match other validators	<i>to be filled</i>
10	Full <code>getCurrentValidators</code> output	For canonical ordering verification	<i>to be provided</i>

Appendix B: Security Considerations

- **BLS Private Keys (`signer.key`):** These are the most sensitive artifacts. They prove validator identity and can sign arbitrary warp messages. Transfer only via SSH/SCP. Delete all copies from the working machine immediately after the registration transaction is confirmed.
- **P-Chain Private Key:** The hex private key used for submitting the P-Chain transaction should be treated as a hot wallet key. Only load the minimum AVAX needed (~11 AVAX).
- **Do NOT commit keys to git.** Add `keys/` to `.gitignore` before starting.

- **Expiry:** The registration message has a 24-hour expiry. If the transaction is not submitted in time, the message becomes invalid. Re-run the script to generate a new one.
- **Weight:** Setting an incorrect weight can affect consensus thresholds. Match existing validator weights unless intentionally rebalancing.

Action Plan Summary

Priority	Action	When
NOW	Run <code>getCurrentValidators</code> to check v6 status	Immediately
1	Scenario A (balance top-up) OR Scenario B (off-chain registration)	Based on pre-check result
2	Fix warp timestamp config on all 4 validators	After v6 is active on P-Chain
3	Sequential restart (one at a time, 2-min gaps)	After config fix applied
4	Verify block production	After all restarts
5	Database resync (if needed, one at a time)	Only if step 4 fails after 5 min

— End of Document —

For questions, contact Ava Labs Solutions Engineering.